Phase 01 — Internet Fundamentals (Complete Notes)

> *How the Internet works, clients & servers, packets, IP addresses, MAC, routers, switches, NAT, ports, TCP/UDP, OSI model.*

---

1. What is the Internet?

The Internet is not a cloud and not magic. It is millions of computers connected by **physical cables** (fiber optic under oceans, copper into buildings) and **radio waves** (WiFi, 4G/5G), all agreeing to speak shared rules called **protocols**.

Two core ideas:

1. **Physical connections** — wires and radios carrying electrical/light signals.
2. **Protocols** — shared rules so any computer can talk to any other, regardless of brand, OS, or country.

The Internet is a **network of networks** (*inter-net*): your home network + office network + Google's network + ISP networks, interconnected. Nobody owns it; there is no central computer.

Why it was created (1960s, ARPANET): expensive university computers couldn't talk to each other, and researchers wanted a network with **no single point of failure** — if one path dies, traffic routes around it. That decentralization still defines the Internet.

2. Client and Server — the most important concept

Every Internet interaction has two roles:

Role	What it does	Examples
Client	*Asks* — always starts the conversation	Browser, mobile app, `curl`, Java `RestTemplate`
Server	*Answers* — sits waiting, listening	NGINX, Spring Boot app, PostgreSQL

⭐ **Key insights most beginners miss:**

- Client and server are **roles, not machines**. A server is not special hardware — it's any computer running a program that *listens*. Your laptop becomes a server the moment you run `mvn spring-boot:run`.
- The same program can be both: your Spring Boot app is a **server** to browsers but a **client** when it calls PostgreSQL.
- **Servers never initiate; clients always start the conversation.**

Analogy — the restaurant 🍔: you (client) order a burger (request); the kitchen (server) is always open, waiting, cooks (processing); the waiter brings food (response). The kitchen never comes to your house asking if you're hungry — and it serves many tables at once (one server, thousands of clients).

3. How data travels: packets

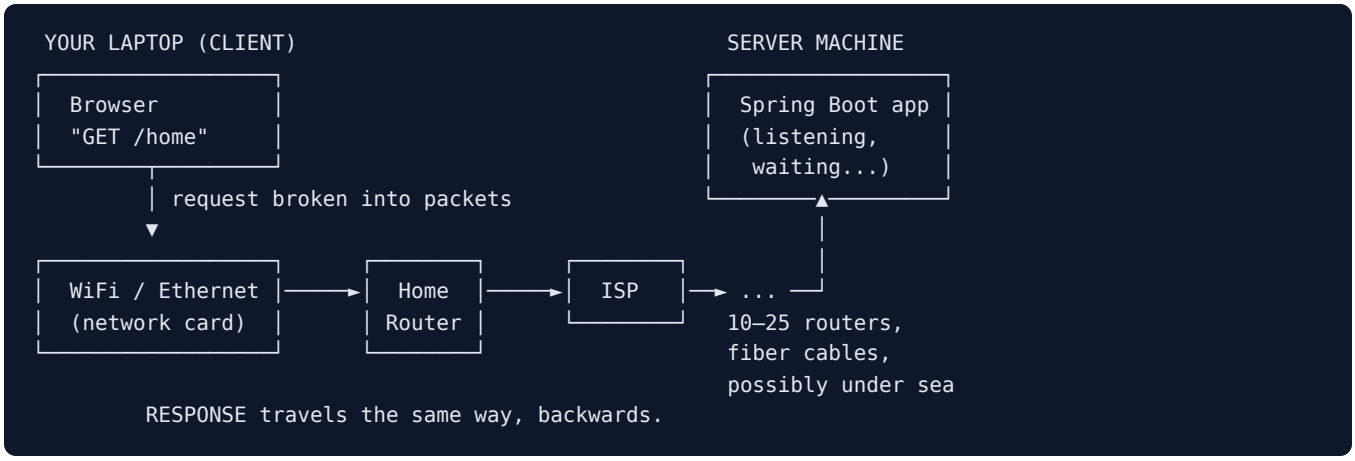
Data is **never sent as one big chunk**. It's chopped into **packets** (~1,500 bytes each), each carrying:

- **Destination address** — where it's going
- **Source address** — where it came from (so the reply knows the way back)
- **Sequence number** — so pieces can be reassembled in order
- A chunk of the actual data

Packets travel independently — they may take **different physical routes** — and are reassembled at the destination.

Analogy — mailing a book 📬: a 500-page book, but the post office only accepts thin envelopes. Send 500 envelopes labeled "page 137 of 500". Some go by truck, some by plane; the receiver reorders and rebuilds. If envelope 137 is lost, only that one is resent (that resend mechanism = TCP, §8).

The journey — architecture diagram



Real-world example: opening `facebook.com` from Dhaka — packets go through your ISP → submarine fiber (SEA-ME-WE cables) → a data center, possibly Singapore → back in ~100–300 ms. Light in fiber ≈ ⅔ light speed → **distance = delay** → why CDNs exist (Phase 8).

4. IP Addresses

An IP address is the unique number identifying a machine on a network — the destination written on every packet. **Analogy:** a postal address for a computer.

IPv4 and IPv6

- **IPv4**: 4 numbers 0–255, e.g. `192.168.0.101` — 32 bits ≈ 4.3 billion addresses. The world has more devices than that → two fixes: **NAT** (§6) and **IPv6**.
- **IPv6**: 128 bits in hex, `2404:6800:4002:81e::200e` — practically unlimited; most systems run both (dual stack).

Public vs Private IP

- **Public IP** — globally unique, reachable from the Internet (your router gets one from the ISP; a VPS/EC2 has one).
- **Private IP** — valid only inside a local network. Reserved ranges (memorize):

```
10.0.0.0    – 10.255.255.255      ← AWS VPCs commonly use this
172.16.0.0  – 172.31.255.255      ← Docker commonly uses this
192.168.0.0 – 192.168.255.255     ← home routers commonly use this
```

Analogy: private IP = apartment number (Flat 4B — meaningless outside the building); public IP = the building's street address.

Localhost

- `127.0.0.1` (= `localhost`) always means **this machine itself**. Traffic never leaves your computer — works with WiFi off; nobody else can open *your* localhost.
- `0.0.0.0` for a *server* means "listen on ALL my interfaces" — accept connections from localhost AND the network. Critical later in Docker and NGINX.

```
ip addr           # your machine's IPs per interface (look for "inet")
curl ifconfig.me  # your PUBLIC IP (as the Internet sees you)
ping 127.0.0.1    # talk to yourself — works offline
```

5. Network interfaces, MAC addresses

- **Interface** = the port through which a machine joins a network: `lo` (loopback/localhost), `eth0` (wired), `wlan0` (WiFi), `docker0` (virtual, Phase 6). One machine can have many, each with its own IP.
- **MAC address** = permanent hardware serial of a network card: `a4:5e:60:d3:8c:1f`. **IP vs MAC:** IP = postal address (changes when you move networks); MAC = national ID (permanent). MAC is used only for delivery *within* the local network (switches); the wider Internet routes on IPs.

6. Routers, switches, NAT

- **Switch** — connects devices within ONE local network, delivers by **MAC** (L2). Analogy: reception desk inside one building.
- **Router** — connects networks to each other, forwards by **IP** (L3) using a routing table; no router knows the full path, only the next best hop. Analogy: inter-city postal sorting hub. Your home "router" box = router + switch + WiFi + DHCP + NAT in one.

NAT — Network Address Translation

Solves IPv4 scarcity: a whole private network shares ONE public IP. The router rewrites outgoing packets and remembers who asked:

```
laptop 192.168.0.101:53000 ──► google.com:443
   router rewrites source →  103.120.5.9:61001   (public IP)
   router notes: 61001 ↔ 192.168.0.101:53000
reply → 103.120.5.9:61001 → router looks it up → forwards to the laptop
```

Analogy: office receptionist — everyone calls out via one office number; she remembers who dialed whom and routes callbacks to the right desk.

Consequence you'll hit in real work: outside machines **cannot initiate** connections to a device behind NAT — that's why your laptop can't serve the world, and why we deploy on VPS/cloud machines with real public IPs (Phase 7).

7. Ports

An IP finds the *machine*; a **port** (1–65535) finds the *program* on it. **Analogy:** IP = building, port = apartment number → `142.250.4.100:443`.

```
22 SSH      80 HTTP      443 HTTPS     53 DNS
25 SMTP     3306 MySQL   5432 PostgreSQL   6379 Redis   8080 Spring Boot default
```

- Ports below 1024 are privileged (need root to listen).
- Only ONE program can listen on a port → the classic `Address already in use` when you start Spring Boot twice.

```
sudo ss -tulpn                # what's listening on which port
sudo lsof -i :8080            # which process owns 8080
kill <PID>
```

8. TCP and UDP

Both deliver packets using IPs + ports; they differ in guarantees.

TCP — Transmission Control Protocol

Connection-based; starts with the **3-way handshake**:

```
client ── SYN ─────► server      "want to talk?"
client ◄─ SYN-ACK ── server      "yes, ready"
client ── ACK ─────► server      "starting"          → connection established
```

Guarantees: **all packets arrive, in order, uncorrupted** — lost ones are re-sent automatically. Cost: slower, more overhead. Used by HTTP/HTTPS, databases, SSH — **everything in this course**. Analogy: registered mail with confirmation + automatic resend.

UDP — User Datagram Protocol

No handshake, no ACK, no ordering — fire and forget. Very fast. Used by video calls, streaming, games, DNS queries (a lost video frame is better skipped than re-sent late). Analogy: postcards.

> *Rule: correctness required → TCP. Speed matters and losses tolerable → UDP.*

9. The OSI model — the map of network layers

```
7  Application   HTTP, DNS, SMTP          ← your Spring Boot lives here
6  Presentation  TLS encryption, encoding
5  Session       connection sessions
4  Transport     TCP / UDP, ports          ← "Layer 4 LB" (AWS NLB)
3  Network       IP, routers               ← packets routed here
2  Data Link     MAC, switches, Ethernet
1  Physical      cables, radio, light
```

Mnemonic (bottom-up): **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way.

Why you care: production vocabulary ("L4 vs L7 load balancer" — Phase 10) and the **debugging ladder**, bottom-up:

1. Cable/WiFi up? (L1–2) → 2. `ping <ip>` (L3) → 3. port open, `nc -zv host 443` (L4) → 4. app answers, `curl` (L7).

(The real Internet uses the simpler 4-layer TCP/IP model: Link → Internet → Transport → Application.)

10. The full picture — one request, everything labeled

```
Browser wants  https://api.example.com/users

[L7] browser builds HTTP request
[L4] TCP: connect to port 443, 3-way handshake, split into segments
[L3] IP: packets stamped  src=192.168.0.101  dst=<server public IP>
[L2] MAC: frame addressed to home router's MAC
[L1] bits over WiFi radio waves
       ▼
Home router: NAT rewrites src → public IP :port, remembers mapping
       ▼
10–25 Internet routers, each forwarding by destination IP
       ▼
Server: public IP, program listening on 443
[L4] TCP reassembles segments in order
[L7] Spring Boot handles the request, responds
       ▼
Response flows back; NAT maps it to the laptop; browser renders
```

If you can narrate this from memory, Phase 1 is yours.

11. Common mistakes

- Thinking "the cloud" isn't physical — AWS = someone else's computers, rented.
- Thinking a server is special hardware — it's a *program that listens*.
- Thinking data goes directly A→B (it hops 10–25 routers) or as one piece (always packets).
- Thinking WiFi is the Internet — WiFi is only the last few meters.
- Confusing IP with MAC (postal address vs national ID; L3 vs L2).
- Thinking teammates can open your `localhost` — use your private IP for that.
- Binding a server to `127.0.0.1` and wondering why the network can't reach it (`0.0.0.0` !). You WILL hit this with Docker.
- Thinking NAT is a security feature (it blocks inbound only as a side effect).
- Saying "TCP is better than UDP" — different problems, different tools.

12. Best practices

- Instinctively run the debugging ladder: `ping` → port check → `curl`.
- Always know: my private IP, my public IP, which ports my apps use.
- Assume the network **will** fail, slow down, drop packets — design with timeouts/retries (later phases).
- Latency is physics: keep servers close to users.

13. Interview questions

1. What happens conceptually when a browser requests a page? (client → routers → server → response)
2. What is a packet, and why split data into packets? *(fair wire-sharing, retransmit only lost pieces, route around failures)*
3. Is "client" a property of a machine or a conversation? Give an example of a program being both.
4. What does a router do? Does it know the full path? Router vs switch — which layer, which address type?
5. IPv4 vs IPv6 — why does IPv6 exist? Which ranges are private?
6. What is NAT? Why can't an outside machine connect to your laptop at home?
7. What is 127.0.0.1? Binding to 127.0.0.1 vs 0.0.0.0?
8. Explain the TCP 3-way handshake. Why does HTTP use TCP but video calls use UDP?

9. Two programs want port 8080 — what happens? How do you find and fix it?
10. Name the OSI layers with one protocol each. What does "L7 load balancer" mean?
11. Why is a US server slower than a Singapore one for a user in Bangladesh, even if both are equally fast?

14. LAB

```
# A. distance = latency
ping -c 4 google.com               # note time=XXms (full round trip)
ping -c 4 facebook.com; ping -c 4 gov.bd     # compare — where are the servers?
traceroute google.com              # every line = one router; line 1 = your home router

# B. identity
ip addr                            # interfaces + private IP
curl ifconfig.me                   # public IP → different! that's NAT
ip route                           # your default gateway = your router

# C. be a client without a browser
curl -v https://example.com        # ">" = request, "<" = response; "Connected" = TCP handshake done

# D. be a server
python3 -m http.server 8000        # terminal 1 — you are now a server
curl http://localhost:8000         # terminal 2 — and its client; watch T1 log it
sudo ss -tulpn | grep 8000         # see it LISTEN
python3 -m http.server 8000        # terminal 3 — FAILS: port taken

# E. reach your machine from another device
# open http://<your-private-ip>:8000 from your PHONE on the same WiFi
# works via private IP; would NOT via localhost. Understand why.
```

15. ASSIGNMENT 01 (submit to Claude)

1. `traceroute` to 3 sites — local (BD), regional (Singapore/India), US (`mit.edu`). Record hop counts + times; explain the differences in 2–3 sentences.
2. Run `ip addr` and `curl ifconfig.me`; paste both and explain why the IPs differ and what NAT did.
3. From memory (no peeking): draw the ASCII journey of a request from browser to Spring Boot server, labeling private IP, public IP, NAT, router, port, TCP.
4. Explain to an imaginary junior (5–6 sentences): what happens when they open a website — using *client, server, packet, router*.
5. Your Spring Boot app must be shown to a teammate on the same office WiFi. What URL do you give, and what must be true for it to work?
6. Why would `server.address=127.0.0.1` break a Dockerized Spring Boot app? (Reason from first principles.)
7. A Spring Boot app calls PostgreSQL and an external payment API. List every client role and every server role.
8. Interview questions 4, 6, 8 — answer from memory in writing.